

Resumo: Adicionando Interação (Eventos no Pygame)

1. Introdução aos Eventos

A interação do usuário com o jogo ocorre através de **eventos**. O Pygame captura todas as ações (cliques, teclas, fechamento de janela) e as envia para o nosso programa através da função `pygame.event.get()`.

Tipos comuns de eventos incluem: - `pygame.QUIT`: Quando o usuário clica no botão de fechar a janela. - `pygame.MOUSEBUTTONDOWN` / `pygame.MOUSEBUTTONUP`: Clique do mouse. - `pygame.KEYDOWN` / `pygame.KEYUP`: Pressionar e soltar uma tecla do teclado.

2. A Fila de Eventos vs `input()`

Diferente da função `input()` clássica do Python, o Pygame **não para a execução** do programa esperando o usuário digitar. - Todas as interações vão para uma **fila de eventos**. - Quando chamamos `pygame.event.get()`, a fila nos devolve os eventos ocorridos na memória e **esvazia a fila** automaticamente. - Se nenhuma tecla foi apertada, a lista retornada simplesmente estará vazia e o jogo continuará rodando normalmente (frame a frame).

3. Lendo Teclas e Atributos de Eventos

Dependendo do tipo do evento, a variável conterá diferentes informações (atributos). - Eventos de teclado (`KEYDOWN`, `KEYUP`) possuem o atributo `.key`. - Eventos de mouse possuem `.pos` (posição) e `.button` (qual botão).

Para saber qual tecla exata foi pressionada, comparamos o atributo `.key` com constantes do Pygame, que sempre começam com `K_`:

```
def recebe_eventos():
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            return False

        # Verifica se o evento é do tipo "tecla pressionada"
        elif evento.type == pygame.KEYDOWN:
            # Verifica qual tecla foi pressionada
            if evento.key == pygame.K_LEFT:
                print('Apertou seta para a esquerda!')
            elif evento.key == pygame.K_RIGHT:
                print('Apertou seta para a direita!')

    return True
```

4. Estado do Jogo e Movimentação

Para movimentar algo na tela, o jogo precisa de um **estado** (por exemplo, um dicionário guardando as coordenadas atuais do jogador). As ações do usuário devem **modificar o estado do jogo**, não redesenhar a entidade diretamente.

Passo 1: Criar o estado Na função `inicializa`, criamos e retornamos o dicionário de estado com posições iniciais:

```
def inicializa():
    # Código pra inicializar (assets, window) ...
    state = {
        'jogador_x': 100,
        'jogador_y': 150
    }
    return window, assets, state
```

Passo 2: Alterar o estado usando eventos A função `recebe_eventos` recebe o `state` e atualiza seus dados com base nas teclas detectadas. (*Lembre-se: o eixo X cresce para a direita e o eixo Y cresce para baixo!*)

```
def recebe_eventos(state):
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            return False

        if evento.type == pygame.KEYDOWN:
            if evento.key == pygame.K_RIGHT:
                state['jogador_x'] += 5 # Move para a direita (positivamente no X)
            elif evento.key == pygame.K_LEFT:
                state['jogador_x'] -= 5 # Move para a esquerda (negativamente no X)

    return True
```

Passo 3: Desenhar usando o novo estado Na função `desenha`, basta usarmos as coordenadas atualizadas dentro do `state` para renderizar o movimento com a imagem e a respectiva posição correta:

```
def desenha(window, assets, state):
    # preenchimentos de fundo ...

    # Desenha o jogador na nova posição
    window.blit(assets['jogador'], (state['jogador_x'], state['jogador_y']))
```